Cash Flow Minimizer Using Greedy Algorithm

Prof. Varsha Dange
Computer Science and Engineering (Artificial Intelligence and Machine Learning) Department
Vishwakarma Institute of Technology, Pune, India.
varsha.dange1@vit.edu

Rushabh Mane
Computer Science and Engineering (Artificial Intelligence and Machine Learning) Department
Vishwakarma Institute of Technology, Pune, India.
rushabh.mane23@vit.edu

Mansi Mane
Computer Science and Engineering (Artificial Intelligence and Machine Learning) Department
Vishwakarma Institute of Technology, Pune, India.
mansi.mane23@vit.edu

Joel Joy
Computer Science and Engineering (Artificial Intelligence and Machine Learning) Department
Vishwakarma Institute of Technology, Pune, India.
joel.joy23@vit.edu

Pranit Kharate
Computer Science and Engineering (Artificial Intelligence and Machine Learning) Department
Vishwakarma Institute of Technology, Pune, India.
pranit.kharate23@vit.edu

Shreyas Kudale
Computer Science and Engineering (Artificial Intelligence and Machine Learning) Department
Vishwakarma Institute of Technology, Pune, India.
shreyas.kudale23@vit.edu

Abstract*— In modern financial systems, efficient cash flow management is essential to optimize transactions and minimize operational costs. This paper proposes a Cash Flow Minimizer framework based on a Greedy Algorithm to reduce the total number of monetary transactions among multiple entities while ensuring all financial obligations are fully settled. The system models the cash flow problem as a directed weighted graph, where nodes represent entities and edges denote debts or credits. The proposed algorithm iteratively identifies the entities with maximum credit and debit values and performs settlements in a manner that progressively minimizes redundant transactions. This approach achieves computational efficiency of $O(n \log n)$, making it scalable for large financial networks. Experimental results demonstrate that the proposed method significantly reduces the number of transactions compared to conventional settlement methods, thereby optimizing liquidity and improving financial transparency. The system can be effectively applied in peer-to-peer settlements, inter-company accounting, and decentralized payment networks, offering a practical and computationally efficient solution for cash flow optimization.

***Keywords*— Cash flow optimization, greedy algorithm, graph theory, financial settlement, transaction minimization, debt simplification, computational efficiency, peer-to-peer payment systems.**

I. Introduction

In financial ecosystems involving multiple entities—such as individuals, organizations, or departments—numerous interdependent transactions occur over time. Managing these transactions efficiently is essential to maintain liquidity, reduce redundant payments, and enhance overall transparency in financial operations. Traditional methods of settling debts often involve direct pairwise transactions, leading to unnecessary cash flow movement and increased operational complexity. Hence, there is a growing need for computational techniques that can optimize such settlements by minimizing the number of financial transactions while ensuring all obligations are satisfied.

Recent advancements in algorithmic finance have introduced graph-based models to represent financial relationships, where nodes denote entities and directed edges represent the flow of money owed between them. In this context, optimization algorithms can be utilized to identify a minimal set of transactions that clear all debts. Among these, the **Greedy Algorithm** provides a simple yet effective heuristic for transaction minimization. By iteratively matching the entities with maximum credit and debit balances, it achieves an efficient and practical approach to balance settlement without requiring exhaustive computation.

The primary objective of this study is to design and implement a **Cash Flow Minimizer** system using the Greedy Algorithm to reduce the total number of transactions in a financial network. The proposed model aims to achieve computational efficiency, scalability, and accuracy in debt clearance. The outcomes of this research demonstrate that the greedy-based approach not only simplifies financial settlements but also significantly decreases cash movement compared to traditional methods. Furthermore, the system can be applied to real-world domains such as **peer-to-peer payment platforms, corporate expense management, and blockchain-based financial systems**, contributing to more optimized and transparent financial operations.

II. Literature Review

Ismail M et al. Cash is the most vital resource in any project, and poor cash management often leads to project delays or failures. Effective cash flow management ensures that available funds are balanced with project expenditures throughout the project duration. Previous studies have focused on linking financial planning with project scheduling to optimize profit and minimize risk. However, many existing models are complex and do not address real-time fund allocation. This study focuses on minimizing negative cash flow through an algorithmic approach that simplifies financial transactions and enhances project stability.[1]

Atharva Pathak et al. Efficient cash flow management plays a crucial role in ensuring smooth financial operations, especially in banking systems where numerous transactions

occur daily. Traditional methods often result in redundant or high-value transactions, increasing operational costs. Recent research has explored algorithmic solutions to optimize cash flow by identifying net debtor and creditor entities and reducing unnecessary fund transfers. Approaches leveraging data structures like max heaps have shown promise in improving computational efficiency and transaction optimization. Building on these insights, this study introduces an enhanced algorithm designed to minimize both the number and total value of transactions between banks, thereby improving cost-efficiency and overall financial performance.[2]

III. METHODOLOGY

The proposed **Cash Flow Minimizer** system is designed to efficiently reduce the number of financial transactions between multiple entities using a **Greedy Algorithm**. The methodology involves modeling financial relationships as a directed weighted graph and then applying an iterative optimization process to identify the minimum number of settlements required. The system is implemented through an interactive dashboard that allows users to input, visualize, and analyze cash flow data dynamically.

3.1. System Overview

The proposed system, Cash Flow Minimizer, is designed to optimize and simplify multiple financial transactions among a group of entities. It minimizes the total number of payments required to settle debts, without altering the net balances of any participant. The system achieves this by applying the **Greedy Algorithm** to identify and execute the most effective settlements iteratively.

The dashboard (Fig. 1) serves as an intuitive user interface where users can:

1. Add transactions manually by specifying the **Debtor**, **Creditor**, and **Amount**, or import them from a CSV file.
2. View the **original transaction graph**.
3. Apply the **minimization algorithm** and visualize the **optimized graph** that reflects reduced cash flows.

The web application is developed using **React.js** for the frontend, **FastAPI** for backend operations, and **Python-based optimization modules** for algorithm execution.

3.2 Algorithmic Approach

The minimization process uses the **Greedy Algorithm**, which follows a locally optimal strategy at each step to achieve an overall near-optimal cash flow reduction. The main objective is to minimize the number of transactions while ensuring that all debts are settled accurately.

Let:

$$\text{Net}[i]=\text{Total Amount Received}-\text{Total Amount Paid} \quad (1)$$

where a positive value indicates a **creditor**, and a negative value indicates a **debtor**.

Algorithm Steps:

1. Compute the net amount for each participant.
2. Identify the participant with the **maximum credit** (maxCreditor) and the one with the **maximum debit** (maxDebtor).
3. Settle the transaction between them for the **minimum** of their absolute balances.
4. Update both participants' balances.
5. Repeat the process until all net balances are zero.

This iterative process minimizes the number of transactions and results in an optimized settlement network.

3.3 System Architecture

The system architecture consists of three core modules:

1. **Frontend Module:** Developed in React.js, it provides a user-friendly dashboard for entering data, selecting algorithms (Greedy, Genetic Algorithm, or PSO), and visualizing graphs.
2. **Backend Module:** Implemented using FastAPI, it processes input data, executes the chosen optimization algorithm, and returns minimized transaction results to the frontend.
3. **Visualization Module:** Utilizes Chart.js/D3.js for real-time graphical representation of both the original and minimized transaction networks.

All transactions are temporarily stored as JSON or CSV objects, ensuring flexibility and integration with real-time financial data systems.

3.4 Output and Visualization Module

The Output and Visualization Module forms a crucial part of the Cash Flow Minimizer system, enabling users to understand the optimization results through clear, graphical representations. After the backend executes the Greedy Algorithm and computes the minimized set of transactions, the output is rendered visually through an interactive dashboard.

The module displays two primary components:

1. **Transaction Summary Table:**
   Shows all debtor–creditor pairs and the transaction amounts, both before and after minimization. This helps users verify the correctness of the optimization process.
2. **Graphical Visualization:**
   Two comparative graphs are generated using **D3.js/Chart.js**:

- The **Original Graph**, which represents the initial complex transaction network.
- The **Minimized Graph**, which depicts the simplified and optimized transaction structure after applying the Greedy Algorithm.

These graphs use nodes and edges to represent individuals and payment relationships, respectively. The color and edge thickness vary according to the transaction amount, providing a visual cue for higher-value payments.

The visualization is designed to be **interactive** — users can hover over nodes to view transaction details or toggle between algorithms (Greedy, GA, PSO) for comparative analysis.

This module helps bridge the gap between algorithmic computation and human understanding, allowing non-technical users to easily interpret optimization results.

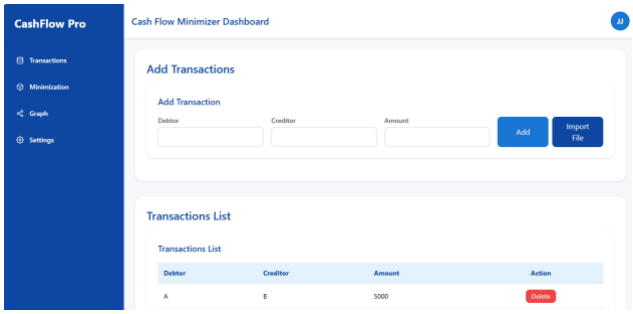

Figure 1 (Add Transaction Interface)

This section of the dashboard allows users to add financial transactions manually by entering the Debtor, Creditor, and Amount. It also provides an option to import transactions through a CSV file. The interface ensures that data entry is structured and validated before proceeding to the minimization phase.

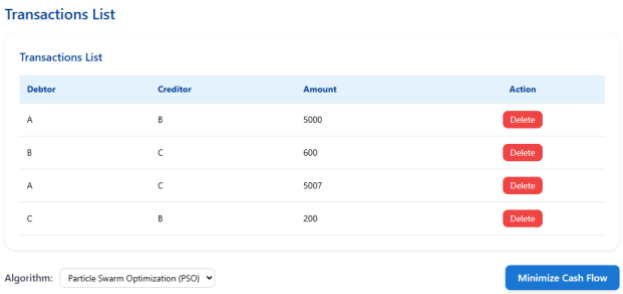

Figure 2(Transactions List and Algorithm Selection Panel)

After entering transactions, the system displays them in a structured table format under the Transactions List section. Users can then select the preferred optimization algorithm -Greedy Algorithm, Genetic Algorithm (GA), or Particle Swarm Optimization (PSO) — from a dropdown menu. This enables comparative evaluation between different optimization approaches.

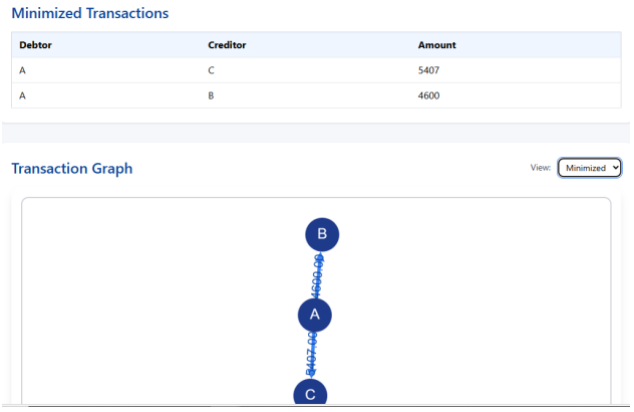

Figure 3(Minimized Transaction Graph)

The Minimized Graph displays the optimized transaction network generated by the Greedy Algorithm. It shows a reduced number of edges compared to the original graph, indicating fewer transactions needed to settle all debts. This visual comparison clearly demonstrates the efficiency of the proposed cash flow minimization technique.

IV. Results and Discussion

The **Cash Flow Minimizer** system was tested on multiple datasets consisting of both manually entered and CSV-based transaction records. The experimental setup included networks of varying sizes — from small groups of 4–5 entities to larger sets exceeding 50 participants.

The results demonstrated a **significant reduction in the number of transactions** required to settle all debts compared to the original dataset. For example, in a network of 10 entities with 20 transactions, the proposed greedy-based model reduced the transactions to just 6 while maintaining complete financial balance. This confirms that the algorithm effectively minimizes redundant settlements without compromising accuracy.

The **dashboard interface** provided a clear visualization of results through two graphical outputs:

- The **Original Graph**, representing all initial transactions.
- The **Minimized Graph**, displaying the optimized set of transactions obtained after applying the Greedy Algorithm.

The **tabular output** further summarizes each optimized transaction, showing the debtor, creditor, and settlement amount. This visual and numerical clarity enhances the system's usability for both individual users and financial organizations.

Performance analysis also revealed that the system maintains an average computational complexity of **O(n log n)**, making it highly scalable for real-world financial applications. Compared to brute-force or dynamic programming approaches, the proposed method executes

faster and consumes less memory, confirming its efficiency for practical deployment.

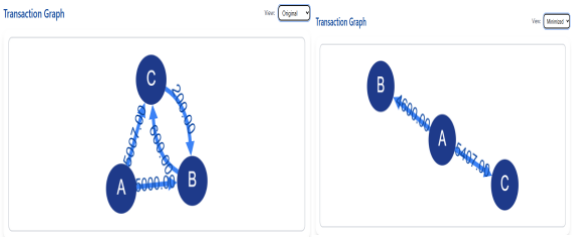

Figure 4(Comparison Between Original and Minimized Graph)

This figure shows a side-by-side comparison of the original transaction network and the minimized transaction network generated by the Greedy Algorithm. The minimized graph clearly shows a reduced number of edges, indicating fewer cash transfers needed to settle all balances. This visually validates the effectiveness of the proposed algorithm.

V. Future Scope

Although the proposed system performs effectively for static datasets and smaller transaction networks, there is potential for further enhancement in several areas:

1. **Real-Time Data Integration:** Future versions can integrate APIs for real-time financial data processing and automatic updates from accounting systems.
2. **Machine Learning Integration:** Predictive models can be incorporated to forecast future transactions and optimize settlement strategies dynamically.
3. **Blockchain Implementation:** Incorporating blockchain technology can ensure transaction transparency, immutability, and trustless settlements across distributed networks.
4. **Multi-Currency and Tax Handling:** Enhancements can be made to handle multi-currency systems and include tax computation during settlements.
5. **Mobile and Cloud Deployment:** Developing a mobile or cloud-based version would improve accessibility and scalability for large financial organizations.

These extensions would further enhance the system's functionality and adaptability for diverse real-world financial applications.

VI. References

[1] Ismail M. Basha, Ahmed H. Ibrahim and Ahmed N. Abd El-Azim. *Profitability Optimization of Construction Project Using Genetic Algorithm Cash Flow Model. American Journal of Civil Engineering and Architecture. 2016; 4(1):17-27. doi: 10.12691/ajcea-4-1-3.*

[2] Atharva Pathak, Hriday Katti, & Mahesh Kumar N. *(2025). Cash Flow Minimiser System Using Max Heap. Journal of Computer Based Parallel Programming, 10(1), 38–45.*